

RAP Cloud 課程 5：Draft (草稿處理)

Lecture

這一課要證明的事

舊 On-Premise 課程完全沒有 Draft 這一課——查證版本演進歷程 ([.claude/rules/sap-adt-mcp.md](#) 第 43 / 56 節) 已經確認：Draft 的完整 Late Numbering / Side Effects 支援要到 On-Premise 7.57 ~ 7.58 (≈S/4HANA 2022 ~ 2023)，而這個專案的 On-Premise 系統卡在 7.54 (1909)，連基礎的 **with draft;** 語法能不能編譯都是問號。這門課的 Cloud 環境版本遠遠超過這個門檻，這是第一次真正能讓學員看到 **Draft** 機制端對端運作。

這一課建立一個全新的獨立實體 (**ZI_RC05_NOTE**)，不延伸 rc02 ~ rc04 用的 **ZI_RC01_TASK**——原因是 Draft 從根本上改變了「CREATE 直接寫入資料庫」這個語意 (一旦啟用 Draft，CREATE 預設會先進 Draft Table，不是直接進正式表)，如果直接在 **ZI_RC01_TASK** 上加 **with draft;**，rc02 ~ rc04 已經驗證過的測試邏輯全部會被打破。

意外的好消息：查證官方文件 ([ABENBDL_DRAFT_TABLE](#) / [ABENBDL_DRAFT_ACTION1_ABEXA](#)) 明講「Draft 完全由 RAP 框架處理，不需要在 ABP 裡寫任何實作」——這是這門課第一次不用碰 Local Types，也就完全繞開了 rc03 / rc04 記錄的 MCP 工具限制 (讀不到 Local Types 內容)。兩個 ABAP Unit 測試一次就全部通過，沒有經過任何除錯，跟舊課程一路踩坑的敘事形成強烈對比。

Draft 解決什麼問題

沒有 Draft 的 Managed RAP BO (rc02 ~ rc04 用的模式)，**create/update** 一旦 **COMMIT ENTITIES** 成功，資料就是正式、對外可見的——這對應「填表單、按下送出，資料立刻生效」的場景。但真實應用常常需要「先開始編輯、中途可以隨時離開不儲存、想清楚了才真正送出」——這正是 Fiori Elements 畫面上「New / Edit」按鈕背後的模型：使用者按下「New」的當下，系統其實是先建立一筆 **Draft (草稿)**，使用者編輯過程中的每一次修改都只影響這份草稿，直到按下「Save」才會真正觸發 **Activate**，把草稿內容轉正、寫進正式資料表；如果使用者中途按「Cancel」，草稿直接 **Discard (丟棄)**，正式資料完全不受影響。

BDEF 語法：**with draft;** + **draft table** + 標準 Draft Action

```
managed implementation in class zbp_i_rc05_note unique;
strict ( 2 );
with draft;

define behavior for ZI_RC05_NOTE alias Note
persistent table zrc05_note
draft table zrc05_note_d
lock master
total etag changed_at
etag master local_changed_at
authorization master ( none )
{
    create;
```

```

update;
delete;

field ( readonly : update ) note_id;
field ( readonly )          changed_at, local_changed_at;
field ( mandatory )         title;

draft action Activate optimized;
draft action Discard;
draft action Edit;
draft action Resume;
draft determine action Prepare;
}

```

跟 rc02 ~ rc04 已經教過的語法相比，新增的部分：

新語法	意義
<code>with draft;</code>	寫在 BDEF Header (<code>strict(2);</code> 之後)，宣告整個 RAP BO 啟用 Draft 處理
<code>draft table zrc05_note_d</code>	緊接在 <code>persistent table</code> 之後，宣告這個 BO 對應的 Draft Table 是哪一張——這張表結構要跟正式表完全一致 (欄位名稱、型別、長度都要對得上)，差別只多一個特殊的 <code>%admin</code> 技術欄位 (見下一節)
<code>total etag changed_at</code>	查證官方文件 (<code>ABENRAP_TOTAL_ETAG_GLOSRY</code>)：這個欄位是給「Draft 轉正式 (Activate)」這個轉換過程用的樂觀鎖定檢查欄位， 位置有規定 ——官方範例明講要緊接在 <code>lock master</code> 之後，這一課照抄這個順序
<code>etag master local_changed_at</code>	rc03 就教過的一般樂觀鎖定欄位 (單一實例的併發修改檢查)，Draft 情境下兩種 etag 並存，各自負責不同階段
<code>draft action Activate optimized; / Discard / Edit / Resume / draft determine action Prepare;</code>	五個標準 Draft Action ，官方文件規定這五個是啟用 Draft 後的必要宣告，全部由框架提供預設實作，這一課完全沒有替它們寫任何 ABP 程式碼

Draft Table 的特殊結構：`%admin` 技術欄位

查證官方文件 `ABENBDL_DRAFT_TABLE` 列出的硬性規則：

- 第一個欄位必須是 `abap.clnt` 型別的 Client 欄位
- 其餘欄位名稱、型別、長度要跟 CDS View 曝露出來的欄位**逐一對應** (這一課沒有替欄位取別名，所以 Draft Table 欄位名稱直接照抄正式表)
- 額外要有一個技術欄位**，用固定寫法 `"%admin" : include sych_bdl_draft_admin_inc;`——這是 SAP 預先定義好的 Include 結構，框架靠它記錄 Draft 的中繼資料 (誰在編輯、什麼時候建立等)，開發者不用 (也不該) 自己去操作這個欄位的內容

```
define table zrc05_note_d {
  key client      : abap.clnt not null;
  key note_id     : abap.char(10) not null;
  title           : abap.char(40);
  content         : abap.char(100);
  changed_at      : abap.utclong;
  local_changed_at : abap.utclong;
  "%admin"        : include sych_bdl_draft_admin_inc;
}
```

官方文件也提到：Eclipse / VS Code 在 BDEF 加上 **draft table <名稱>** 之後，如果這張表還不存在，編輯器理論上會提供「快速修正」自動產生正確結構——這一課因為擔心這個快速修正在 VS Code 這個特定擴充套件上不一定可靠，改用比較保險的做法：先手動建好表格空殼，再由 **Claude** 寫入包含 **%admin** 的完整 **DDL**，兩種做法效果相同。

第一個情境：Create → Activate

```
" ---- CREATE 一筆 Draft 實例 ----
MODIFY ENTITIES OF zi_rc05_note
  ENTITY Note
  CREATE FROM
    VALUE #( ( %cid = 'C1'
               %is_draft = if_abap_behv=>mk-on
               %control-note_id = if_abap_behv=>mk-on
               note_id          = lv_id
               %control-title   = if_abap_behv=>mk-on
               title            = 'Draft Test Note'
               %control-content = if_abap_behv=>mk-on
               content          = 'Initial content' ) )
  FAILED DATA(ls_failed_create)
  REPORTED DATA(ls_reported_create).

COMMIT ENTITIES RESPONSE OF zi_rc05_note ...
```

- **%is_draft = if_abap_behv=>mk-on**——查證官方文件 **ABAPDERIVED_TYPES_IS_DRAFT : %is_draft** 是 BDEF 衍生型別 **%tky** (技術鍵) 的一個組成部分，**要建立 Draft 實例**，這個旗標必須明確設成 **mk-on**；如果整句 CREATE 完全不提 **%is_draft**，預設行為是直接建立正式 (Active) 實例，跳過 Draft——這正是這一課第二個情境 (**edit_and_discard**) 會用到的另一種寫法。
- **%control-<欄位> = if_abap_behv=>mk-on**——這一課第一次出現的新概念：**%control** 是另一組衍生型別，每個欄位都有一個對應的旗標，用來明確告訴框架「這個欄位是使用者真的要設定的值」，區分「沒有填寫」跟「填寫成初始值」這兩種不同意圖。rc02 ~ rc04 因為 BDEF 沒有啟用 Draft，直接指派欄位值就會被框架當作「有填寫」；**Draft** 情境下編輯階段可能分好幾次局部更新同一筆草稿，框架需要更精確知道「這次呼叫到底動了哪些欄位」，所以要求明確用 **%control** 標註。
- 建立完之後 (**COMMIT ENTITIES** 成功)，直接下 **SQL** 查兩張表驗證：**zrc05_note_d** (Draft Table) 查得到這筆資料，**zrc05_note** (正式表) 完全查不到——證明 CREATE + **%is_draft=mk-on** 真的只進了

Draft Table °

```
" ---- EXECUTE Activate : 把草稿轉正 ----
MODIFY ENTITIES OF zi_rc05_note
  ENTITY Note
  EXECUTE Activate
    AUTO FILL CID
    WITH VALUE #( ( %key-note_id = lv_id ) )
  FAILED DATA(ls_failed_activate)
  REPORTED DATA(ls_reported_activate).

COMMIT ENTITIES RESPONSE OF zi_rc05_note ...
```

- **AUTO FILL CID**——這是 **Activate** / **Edit** 這兩個 Draft Action 呼叫時固定會用到的新語法，官方 ABEXA 範例明講的寫法；**Prepare** / **Discard** 則不用（直接 **FROM VALUE #(...)**）。查證文件沒有明講兩者差異的確切原因，合理推測是 **Activate** / **Edit** 這兩個操作在框架內部會產生新的追蹤識別（Draft→Active 或 Active→Draft 的映射關係），需要框架自動補上 **%cid**；**Prepare** / **Discard** 純粹在 Draft Table 內部操作，不需要。這一課先照官方範例的固定寫法使用，具體原因留給之後深入探索。
- **Activate** 成功之後，再次查兩張表：這次 **zrc05_note**（正式表）查得到、**zrc05_note_d**（Draft Table）變回空的——完整驗證了「Draft → Active」的轉換。

第二個情境：Edit → Discard

```
" ---- CREATE 一筆正常的 Active 資料 ( %is_draft = mk-off , 跳過 Draft ) ----
MODIFY ENTITIES OF zi_rc05_note
  ENTITY Note
  CREATE FROM
    VALUE #( ( %cid = 'C2'
              %is_draft = if_abap_behv=>mk-off
              ... ) )
  ...
```

刻意示範另一種 CREATE 寫法：**%is_draft = if_abap_behv=>mk-off** 明確表示「這次不要走 Draft，直接建正式資料」——這是 rc02 ~ rc04 那種「沒有 Draft」情境的等效寫法，證明啟用 **with draft**；之後，舊有的直接寫入模式並沒有被廢除，只是多了一種新選項。

```
" ---- EXECUTE Edit : 把 Active 複製一份到 Draft Table ----
MODIFY ENTITIES OF zi_rc05_note
  ENTITY Note
  EXECUTE Edit
    AUTO FILL CID
    WITH VALUE #( ( %key-note_id = lv_id
                  %param-preserve_changes = 'X' ) )
  FAILED DATA(ls_failed_edit)
  REPORTED DATA(ls_reported_edit).
```

Edit 對應 Fiori Elements 畫面上「編輯」按鈕的動作：把一筆已經是正式資料的實例，複製一份進 Draft Table，讓使用者可以安心修改而不影響正式資料，直到之後呼叫 **Activate** 才會真正覆蓋回去。`%param-preserve_changes` 是 **Edit** 這個框架內建 Action 自帶的輸入參數（不是我們自己宣告的），控制「如果這個 Key 剛好已經有其他未處理完的草稿，要不要保留」，這一課直接照官方範例給 'X'。

```
" ---- EXECUTE Discard : 丟棄這份 Draft 複本 ----
MODIFY ENTITIES OF zi_rc05_note
  ENTITY Note
  EXECUTE Discard
    FROM VALUE #( ( %key-note_id = lv_id ) )
  FAILED DATA(ls_failed_discard)
  REPORTED DATA(ls_reported_discard).
```

Discard 對應「取消編輯」——驗證重點：**Discard** 之後，`zrc05_note_d` 這筆複本消失，但 `zrc05_note`（正式表）裡的原始資料**完全沒被動過**（`title` 還是最初的 'Original Title'，不是編輯階段可能被改到的任何值——雖然這一課的測試沒有在 Edit 之後真的改欄位就直接 Discard，但已經足以證明「Discard 前 Draft 的任何存在都不影響正式資料」這個核心保證）。

ABAP Unit 執行結果

```
Unit Test Results for CLAS/I ZCL_RC05_NOTE_TEST.main
Status: ALL TESTS PASSED
Total: 2 | Passed: 2 | Failed: 0

[PASS] ZCL_RC05_NOTE_TEST
[PASS] CREATE_DRAFT_AND_ACTIVATE (0.850s)
[PASS] EDIT_AND_DISCARD (0.250s)
```

兩個測試一次就通過，沒有經過任何除錯——這是這門課從 rc01 以來第一次沒有踩坑的一課，直接印證了官方文件「Draft 完全由框架處理」這句話的分量：只要 BDEF 語法寫對、Draft Table 結構符合規定，框架把 Create / Activate / Edit / Discard 這一整套複雜的狀態轉換都處理得妥妥貼貼，完全不需要手寫任何 ABP 邏輯。

學習目標

- 能講出 Draft 解決的問題：讓「開始編輯」跟「真正送出」分開，中途可以隨時放棄不影響正式資料，對應 Fiori Elements 的 New / Edit / Save / Cancel 使用者體驗
- 能寫出啟用 Draft 的 BDEF 語法：`with draft;` (Header) + `draft table <名稱>` (緊接 `persistent table` 之後) + 五個標準 Draft Action (`Activate optimized` / `Discard` / `Edit` / `Resume` / `draft determine action Prepare`)
- 知道 Draft Table 的結構規則：欄位要跟正式表逐一對應、第一欄位是 Client、額外要有 `"%admin"` : `include sych_bdl_draft_admin_inc`; 技術欄位
- 能講出 `total etag` 跟 `etag master` 的差異：前者用在 Draft→Active 轉換時的併發檢查、後者是一般的單實例併發檢查
- 能寫出 `%is_draft` / `%control` 在 CREATE 時的用法：`%is_draft=mk-on` 建立草稿、`%is_draft=mk-off` (或省略) 直接建正式資料；`%control-<欄位>=mk-on` 明確標註「這個欄位真的要設定」

- 能講出 `Activate` / `Edit` / `Discard` 三個 Draft Action 各自的作用，並知道 `Activate` / `Edit` 呼叫要加 `AUTO FILL CID`，`Discard` (跟 `Prepare`) 不用
- 知道這一課完全不需要寫任何 ABP 實作 (`ZBP_I_RC05_NOTE` 是純空殼)，Draft 的完整生命週期都是框架內建行為

物件清單

物件	名稱	型別
正式資料表	<code>ZRC05_NOTE</code>	TABL/DT
Draft 資料表	<code>ZRC05_NOTE_D</code>	TABL/DT
CDS Root View Entity	<code>ZI_RC05_NOTE</code>	DDLS/DF
Managed Behavior Definition (<code>with draft</code>)	<code>ZI_RC05_NOTE</code>	BDEF/BDO
Implementation Class (純空殼，無 Local Types 內容)	<code>ZBP_I_RC05_NOTE</code>	CLAS/OC
ABAP Unit 測試類別	<code>ZCL_RC05_NOTE_TEST</code>	CLAS/OC

套件：`ZRAPCLOUD`。全部物件由使用者在 Eclipse 建立空殼，Claude 用 MCP 寫入完整內容並驗證——這一課因為不需要碰 Local Types，是目前為止使用者手動步驟最少的一課。

驗證方式

1. `get_abap_diagnostics` 確認全部物件無語法錯誤
2. `abap_activate` 全部回報 `Activation successful`
3. `run_unit_tests` 對 `ZCL_RC05_NOTE_TEST` 執行，`ALL TESTS PASSED` (2/2，見上方完整輸出)
4. 兩個測試方法內部都直接下 SQL 查 `zrc05_note` / `zrc05_note_d` 兩張表，獨立於 RAP 框架之外驗證資料真的在該在的表裡

思考題

1. `total etag` / `etag master` 這一課各自對應一個獨立欄位。如果只宣告 `etag master`、完全不宣告 `total etag`，你預期 BDEF 啟用時會發生什麼事？(提示：這一課沒有實測這個情境，可以自己在 Eclipse 試著拿掉看看)
2. `Edit` 呼叫的 `%param-preserve_changes` 這一課固定給 `'X'`。查官方文件 `ABENBDL_DRAFT_ACTION` 或相關 Action 說明，這個參數給 `abap_false` (空值) 會有什麼不同行為？
3. 這一課的 `Prepare` (`draft determine action Prepare;`) 宣告了但完全沒有掛任何 `Determination` / `Validation` (對照官方 `ABENBDL_DRAFT_ACTION2_ABEXA` 範例，那裡的 `Prepare` 掛了 `determination(always) setCharField;`)。如果要在這一課的 `Note` 實體上，讓 `Prepare` 觸發 rc03 教過的 `Validation` (例如檢查 `title` 不可以是特定關鍵字)，BDEF 跟 Local Types 大概要怎麼改？(提示：這會讓這一課重新踩到 Local Types 的 MCP 工具限制，需要使用者在 Eclipse 貼上)
4. 官方文件提到 Draft Table 「不建議用 ABAP SQL 直接存取，只建議透過 RAP 框架 (EML) 存取」，但這一課的測試偏偏用 SQL `SELECT` 驗證兩張表的內容。為什麼「唯讀 `SELECT`」是安全的，官方文件真正警告的是什麼操作？(提示：回顧 `ABENBDL_DRAFT_TABLE` 裡示範「直接用 SQL `DELETE` 清空 Draft Table 會殘留中繼資料」的那個範例)

答案

見 `zrc05_note.tabl.abap` (正式表)、`zrc05_note_d.tabl.abap` (Draft 表, 含 `%admin` 欄位)、`zi_rc05_note.ddls.abap` (CDS View)、`zi_rc05_note.bdef.abap` (含 `with draft` 完整內容)、`zbp_i_rc05_note.clas.abap` (純空殼)、`zcl_rc05_note_test.clas.abap` (兩個測試方法)。SAP 端物件套件 `ZRAPCLOUD`, `run_unit_tests` 執行結果: `ALL TESTS PASSED (2/2)`。